

UNIT - I

Introduction to Algorithm Analysis, Space and Time Complexity analysis, Asymptotic Notations. AVL Trees – Creation, Insertion, Deletion operations and Applications
Heap Trees (Priority Queues) – Min and Max Heaps, Operations and Applications

Introduction:

An algorithm is a description or statement of sequence of activities that constitute a process of getting desired outputs from the given inputs. All algorithms must satisfy the following criteria.

1. Input: Zero or more inputs will be supplied externally.
2. Output: At least one quantity will be produced.
3. Definiteness: Each instruction should be clear and unambiguous.
4. Finiteness: The algorithm should terminate after a finite number of steps.
5. Effectiveness: The instructions are very much simple to implement and not depended on any external knowledge to understand.

Key factors for designing an algorithm:

1. How to design an algorithm?
2. How to validate an algorithm?
3. How to analyze an algorithm?
4. How to test an algorithm?

Ex 1:

Algorithm to find sum of digits in a number.

Step 1 : Start

Step 2 : Input 'N'

Step 3: While (N!=0)

 Rem := N%10

 S := S + (Rem*10)

 N := N/10

Step 4: Print S

Step 5: Stop

Ex 2:

Algorithm SumofIndividualDigits

Step 1: Input N

Step 2: Sum = 0

Step 3: While (N != 0)

Rem = N % 10;

Sum = Sum + Rem;

N = N / 10;

Step 4: Print Sum

Techniques to represent an Algorithm:

Specifying algorithms are done by two ways;

1. Pseudo code Convention
2. Recursive algorithms

Pseudo code Convention:

Algorithms are described in natural language like English. Graphical representations are called as flowcharts. Algorithms are different from a program. An algorithm consists of a name and a body.

Rules:

1. An Algorithm is a procedure which consists of heading and body.
2. Comment line begins with //.
3. The compound statements should be in between { }.
4. Identifiers should begin with a letter. The data types of variables are not explicitly declared.
5. Statements are delimited by a ;
6. Assignment of values to variables are done by using :=.

Ex: a:=10;

7. For producing true or false values, the logical operators and, or and the relational operators <, <=, >, >= and # are used.
8. Elements of array are accessed using [].
9. For, while, repeat..until loops are used for looping.
 - a. for var:= v1 to v2 step <<stepvalue>> do

{

Statement block;

}

Here V1, V2 and step value are the arithmetic expressions. V1 and V2 are the integer or real or numeric constants. The clause step value is optional, if not specified it will be taken as +1.

Ex: Algorithm for Sum of 'n' numbers.

b. while (condition) do

{

Statement block;

}

The statement block gets executed until the given condition is true. The condition is evaluated at the beginning of the loop.

Ex: Algorithm for Sum of 'n' Even numbers.

c. repeat

{

Statement block;

} until (condition);

The statement block gets executed as long as the given condition is false. The condition is evaluated after executing the statement block.

Ex: Algorithm for finding factorial of a given number.

10. A conditional statement has the following forms;

a. If (condition) then { Statements; }

b. If (condition) then { Statements; } else { Statements; }

11. Inputs and outputs are done using read and write.

Algorithms can be specified as follows;

Algorithm name (parameter list)

{

Statements;

}

Where,

Name specifies the name of the procedure and parameter list is a list of parameters. The body of the algorithm is having one or more statements enclosed within curly braces.

Ex: algorithm for finding and returning maximum value from 'n' given numbers.

1. Algorithm max(A, n)
2. {
3. Result = A[1];
4. for i := 2 to n do
5. If A[i] < Result then Result := A[i];
6. Return Result;
7. }

Recursive Algorithms:

An Algorithm calls itself is said to be recursion.

Types:

1. **Direct recursion:** An Algorithm can call themselves is said to direct recursion.

Ex: int fun(int x)

```

{
    if (x<=0)
        return x;
    else
        return (fun(x-1));
}
```

2. **Indirection recursion:** Algorithm A is said to be indirect recursive if it calls another algorithm which in turn call A.

Ex: int fun(int x)

```

{
    if(x<=0)
        return x;
    return (fun1(x-1));
}

int fun1(int y)
{
    return fun(y);
}
```

Advantages of pseudo code conventions over flow charts:

The advantage of pseudo code over flowchart is that;

- a. It is very much similar to the final program code.
- b. It requires less time and space to develop
- c. We can write it in our own way as there are no fixed rules.
- d. It can be read and understood easily.
- e. Converting a pseudocode to programming language is very easy as compared with converting a flowchart to programming language.

Disadvantages :

- ➔ It is not visual.
- ➔ We do not get a picture of the design.
- ➔ There is no standardised style or format, so one pseudo code may be different from another.
- ➔ For a beginner, It is more difficult to follow the logic or write pseudo code as compared to flowchart.

Techniques to Design an Algorithm”:

1. Divide and Conquer:

It follows top-down approach to solve a problem. This technique solves problems in 3 steps. They are;

- a. Divide: In this step the original problem is divided into sub problems.
- b. Conquer: In this step every sub problem is solved individually.
- c. Combine: In this step all the sub problems are merged to get the solution of the original problem.

2. Greedy Technique:

This method is used to solve an optimization problem. This algorithm does not always guarantee the optimal solution but it generally produces solutions that are very close to the optimal value

3. Dynamic Programming:

It is similar to divide and conquer but it follows bottom-up approach to solve a problem. In dynamic programming the result is obtained from the smaller sub problems are reused in the calculation of larger sub problems. So it avoids re- computations, which reduces the time while solving a problem.

4. Backtracking:

Backtracking follows Depth First Search from the possible set of solutions. During search, if an alternative doesn't work, the search backtracks and tries with next alternative.

5. Branch and Bound:

It is a general optimized technique, that applies where the Greedy and Dynamic Programming fails. Branch and Bound uses Breadth First Search. Branch and Bound is the most widely used tool for solving large scale NP Hard optimization problems.

Performance Analysis:

Algorithm analysis refers to the task of determining computing time and storage space requirements of an algorithm. It is also known as performance analysis. It enables us to select an efficient algorithm. The analysis can be computed in two phases. They are;

- 1. Priori analysis:** This is the theoretical analysis of an algorithm. Efficiency of an algorithm is measured by assuming that the factors like processor speed is constant and has no effect on implementation.
- 2. Posteriori analysis:** The selected algorithm is implemented in any programming language and is executed on target machine. In this analysis actual time and space requirement are collected. It is also known as empirical analysis.

Performance of an algorithm can be measured in the following ways;

- Space Complexity
- Time Complexity
- Amortized Complexity
- Asymptotic Notation
- Performance Measurement

Space Complexity:

When we design an algorithm, space is required to;

- Store program instructions
- Store constant values
- Variable values

Space Requirement:

- **Instructional Space:** The memory which is used to store compiled version of instructions.
- **Environmental Stack:** The memory which is used to store the details about partially executed functions.
- **Data Space:** The memory which is used to store variables and constants.

$$S(p) = C + Sp \text{ (instance characteristics)}$$

Where,

C= Constant

Sp = Variable Space

The space needed by each algorithm is the sum of the following components;

- a. **Fixed Part:** It is independent of number and size of inputs and outputs. This part typically includes instruction space, space of simple variables, fixed-size components and space for constants, etc.
- b. **Variable Part:** It consists of space needed by component variable, reference variables and the recursion stack space.

The memory required by an algorithm is increased when input is increased, then the space complexity is said to be linear space complexity.

Ex 1:

1. **Algorithm** Sum (a, b, c)
2. {
3. return (a+b+c);
4. }

If each variable needs one word then $S(P) = 3$.

Space Complexity = $O(1)$

Ex 2:

1. **Algorithm** Sum (a, n)
2. {
3. s := 0.0;
4. **for** i := 1 **to** n **do**
5. s:= s + a[i];
6. **return** s;
7. }

Space for the algorithm is;

Size of variable n	-	1 word
Size of array a	-	n words
Loop variable i	-	1 word
Sum variable s	-	1 word

Total = n+3 words

Therefore $S(p) \geq (n+3)$

Space Complexity = $O(n)$

Ex 3: Recursion

1. **Algorithm** Rsum(a,n)
2. { if (n<=0) then return 0.0;
3. Else return rsum(a, n-1) + a[n];
4. }

Space for the algorithm is;

Return address-	1 word
Pointer to a	- 1 word
Local variable n	- 1 word

Each recursion needs 3 words.

Depth of recursion is n+1.

Therefore $S(p) \geq 3(n+1)$

Time Complexity:

Time complexity of an algorithm is the amount of computing time it needs to run to complete its task. It is the sum of compile time and execution time.

Compile Time:

- Does not depends on instance characteristics
- Also once compiled program will be run several times without recompilation

Execution Time:

- Depends on particular problem instance i.e. inputs and outputs.

$$T(p) = C + t_p$$

Where,

C = Compile Time T_p = Run Time

Calculation of Time Complexity:

1. Count the number of steps in each iteration in the program execution.
2. Count only executable statements.

Ex:

Comments	-	0 steps
Assignment	-	1 step
Condition	-	1 step
Loop	-	(n+1) steps
Body of the loop	-	n steps

The amount of time required by an algorithm is increased when input is increased, then the time complexity is said to be linear time complexity.

Ex 1:

St.No.	Statement	S/E	Frequency	Total Steps
1	Algorithm Sum(a, n)	0	-	0
2	{	0	-	0
3	S:=0;	1	1	1
4	for i :=1 to n	1	N+1	N+1
5	S:=S + a[i];	1	N	N
6	return S;	1	1	1
7	}	0	-	0
Total Number of Steps:				2N+3

S/E - 0 – Not Executed

1 - Executed

Ex 2:

St.No.	Statement	S/E	Frequency	Total Steps
1	Algorithm Add(a, b, c, m, n)	0	-	0
2	{	0	-	0
3	for i :=1 to m	1	M+1	M+1
4	for j:=1 to n	1	M(N+1)	MN+M
5	C[i][j]:= a[i][j] + b[i][j];	1	MN	MN
6	}	0	-	0
Total Number of Steps:				2MN+2M+1

Ex 3 : Recursion

St.No.	Statement	S/E	Frequency		Total Steps	
			N=0	N>0	N=0	N>0
1	Algorithm Rsum(a, n)	0	-	-	0	0
2	{	0	-	-	0	0
3	if (n<=0) then	1	1	1	1	1
4	return 0;	1	1	0	1	0
5	else	0	0	-	0	0
6	return Rsum(a, n-1) + a[n];	1	0	1+X	0	1+X
7	}	0	-		0	0
Total Number of Steps:					2	2+X

$T(n) = 2$ for $(n=0)$ and $T(n) = 2+X$ for $(n>0)$

Exercise :

Write an algorithm to find the sum of first n integers and derive its time complexity.

Order of growth:

It describes how running time or space requirement of an algorithm increases as the input size 'n' increases. It is represented using asymptotic notations.

The following are the common order of growth functions.

S.No.	Function	Name
1	$O(1)$	Constant
2	$O(\log n)$	Logarithmic
3	$O(N)$	Linear
4	$O(N \log n)$	$N \log n$
5	$O(N^2)$	Quadratic
6	$O(N^3)$	Cubic
7	$O(2^n)$	Exponential
8	$O(N!)$	Factorial

Asymptotic Notations:

Time complexity is represented by using asymptotic notations. It calculates;

- **Best Case Complexity:** In this case the time / space required by a specific algorithm is low.

Ex: Sort the numbers 1 2 3 4 5 6 in ascending order

Here, all the numbers are already in ascending order, hence the algorithm will take the minimum time.

- **Average Case Complexity:** In this case the time / space requirement is more than the best case but less than the worst case for the specific algorithm.

Ex: Sort the numbers 1 2 3 5 4 6 in ascending order

Here, few numbers are already in ascending order and few numbers are unsorted; hence the algorithm will take more time when compared to best case.

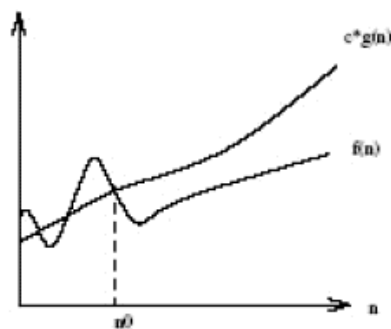
- **Worst Case Complexity:** In this case the time / space requirement is more than the averages case.

Ex: Sort the numbers 6 5 4 3 2 1 in ascending order

Here, all the numbers are unsorted, hence the algorithm will take maximum time.

Big-Oh (O): It is a method of representing upper bound of algorithms running time. (Worst Case)

Definition: Let f and g are nonnegative functions. The function $f(n) = O(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \leq c * g(n)$ for all $n, n \geq n_0$.



Big-oh expressions do not have constants or low-order terms. The idea of $f(n)$ is the exact complexity of an algorithm as a function of the problem size 'n' and that $g(n)$ is the upper-bound on that complexity. Number of upper bounds may be found but take the least upper bound $g(n)$ among all the upper bounds.

Example 1: $f(x) = 2x^2 + 2x + 1$

Step1: Identify all the terms

- $2x^2$ – Quadratic Term
- $2x$ – Linear Term
- 1 – Constant Term

Step 2: Find the dominant term

- In Big Oh analysis, we consider the term that grows the fastest as x becomes very large.
- $2x^2$ grows faster than $2x$ and 1 .

Step 3: Remove the constants

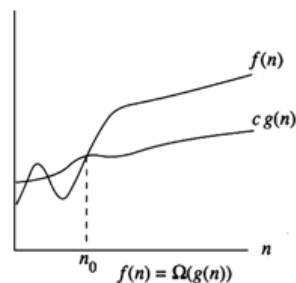
- Big Oh ignores constant multipliers because they do not significantly affect growth rate.
- $2x^2 \rightarrow x^2$

Step 4:

- The Big Oh notation is; $O(x^2)$

Omega (Ω): This is used to represent the lower bound of an algorithms running time. It denotes shortest amount of time taken by the algorithm. (Best Case)

Definition: Let f and g are nonnegative functions. The function $f(n) = \Omega(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \geq c * g(n)$ for all $n, n \geq n_0$.



Example 1: $f(x) = 2x^2 + 2x + 1$

Step1: Identify all the terms

- $2x^2$ – Quadratic Term
- $2x$ – Linear Term
- 1 – Constant Term

Step 2: Big Omega represents the lower bound of the function. It tells the minimum growth rate of the algorithm. It is x .

Step 3: Find the dominant term

- In Big Omega analysis, we consider the term that grows the fastest as x becomes very large.
- $2x^2$ grows faster than $2x$ and 1 .

Step 3: Remove the constants

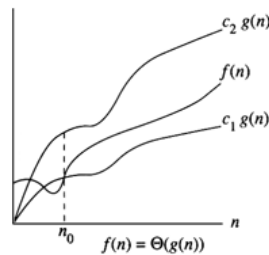
- Big Omega ignores constant multipliers because they do not significantly affect growth rate.
- $2x^2 \rightarrow x^2$

Step 4:

- The Big Omega notation is; $\Omega(x^2)$

Theta (Θ): This is used to represent the average bound of an algorithms running time. It denotes average amount of time taken by the algorithm. (Average Case)

Definition: Let f and g are nonnegative functions. The function $f(n) = \Theta(g(n))$ iif there exist positive constants c_1, c_2 and n_0 such that $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$ for all $n, n \geq n_0$.



Example 1: $f(x) = 2x^2 + 2x + 1$

Step1: Identify all the terms

- $2x^2$ – Quadratic Term
- $2x$ – Linear Term
- 1 – Constant Term

Step 2: Big Theta represents the tight bound of the function. It means the function grows at the same rate in both upper bound and lower bound.

Step 3: Find the dominant term

- In Big theta analysis, as x becomes very large x is dominated by $2x^2$.

Step 3: Remove the constants

- Big theta ignores constant multipliers because they do not significantly affect growth rate.
- $2x^2 \rightarrow x^2$

Step 4:

- The Big Theta notation is; $\Theta(x^2)$

Little-Oh (o):

It is a theoretical measure of the execution of an algorithm, usually the time or memory needed, given the problem size n , which is usually the number of items.

Definition: Let f and g are nonnegative functions. The function $f(n) = o(g(n))$ iff $f(n) < c * g(n)$ for any positive constants $C > 0$ and $n_0 > 0$ and $n > n_0$.

Big Oh is the set of all functions with a smaller or the same order of growth $f(n)$.

Ex: $O(n^2) = \{n^2, 100n+5, \log n, \text{etc.....}\}$

Little Oh is the set of all functions with a smaller rate of growth $f(n)$.

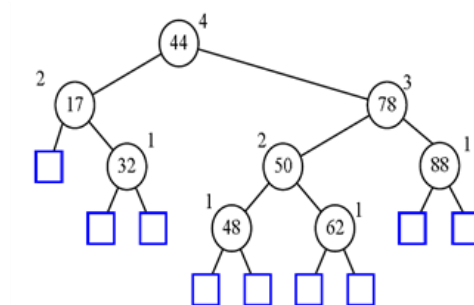
Ex: $o(n^2) = \{100n+5, \log n, \text{etc.....}\}$

AVL Trees:

In 1962 Adelson, Velsky and Landis introduced AVL trees, which are height balanced binary search trees. AVL trees are self-balanced binary search trees. In an AVL tree, each node has a balance factor. This balance factor is the difference height between left and right subtrees. The balance factor can be only either -1 or 0 or 1.

Balance of the node = height of left sub tree – height of right sub tree

For an AVL tree the balance of the node is always -1 or 0 or +1.



Properties of AVL Tree:

- **Balanced Tree:** An AVL tree is self-balanced tree. This means that the heights of the left and right subtrees of any node differ by at most one.
- **Height-Balanced Property:** The difference in height (also called the balance factor) between the left and right subtrees of any node is -1, 0, or 1.

- **Self-Balancing:** An AVL tree automatically performs rotations to maintain balance after every insertion and deletion.
- **Binary Search Tree Properties:** An AVL tree follows the properties of a binary search tree (BST). This means that for any node, the left subtree contains only nodes with values less than the node's value, and the right subtree contains only nodes with values greater than the node's value.
- **Rotations:** To maintain balance, an AVL tree uses four types of rotations: left rotation, right rotation, left-right rotation, and right-left rotation.

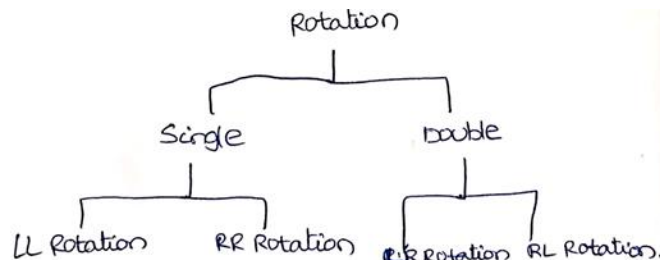
Operations of AVL Tree:

The following are the operations that can be performed on an AVL Tree.

1. Insertion
2. Deletion
3. Searching

Rotations:

After performing insertion or deletion operations, the balance factor of an AVL tree may become violated. To maintain the Balance Factor, the tree must be restructured. For restructuring the AVL tree, the following rotations are used:

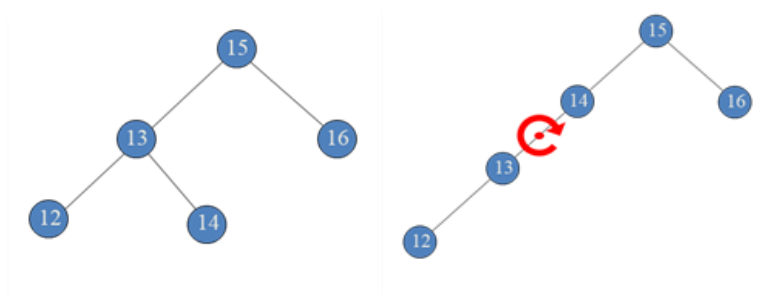


LL Rotation: LL rotation is required whenever BF is violated after inserting a new node into left child of left subtree.

Steps:

1. Identify the node where BF is violated.
2. Identify the left child of the violated node and make it as root node.
3. BF violated node will be the right node and new node will become left node.

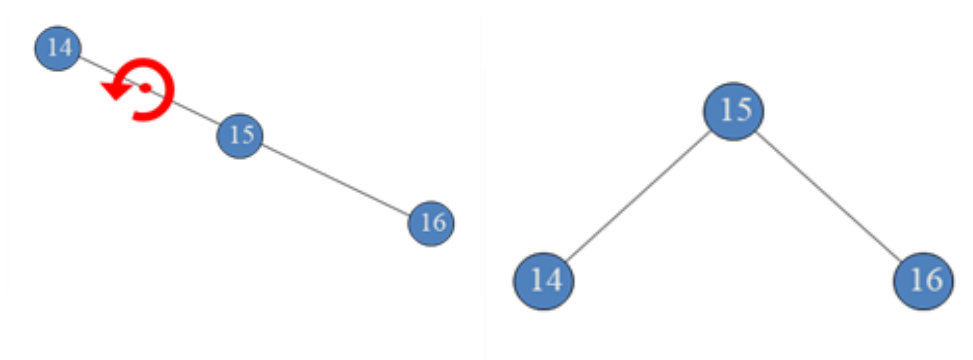
Ex:



RR Rotation: RR rotation is required whenever BF is violated after inserting a new node into right child of right subtree.

Steps:

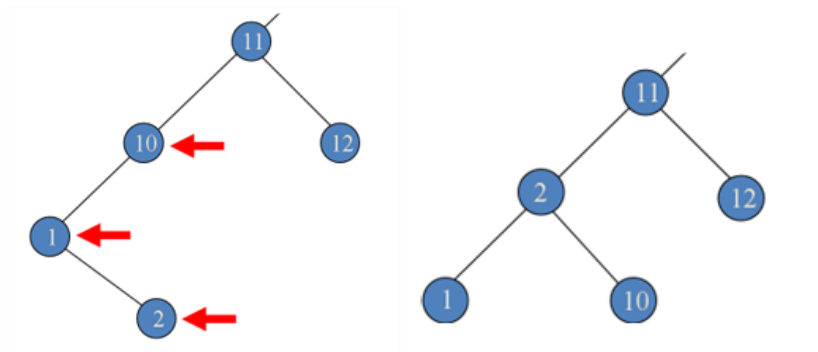
1. Identify the node where BF is violated.
2. Identify its parent node and make it as root node.
3. The root node becomes left of the new root and the effected node becomes right of the new node.



LR Rotation: LR rotation is required whenever the BF is violated after inserting new node as right node of left subtree.

Steps:

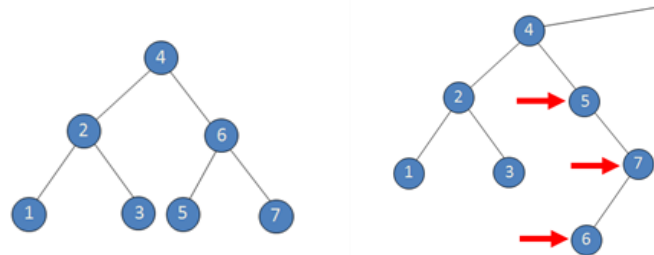
1. Perform RR Rotation
2. Perform LL Rotation



RL Rotation: RL rotation is required whenever the BF is violated after inserting new node as left node of right subtree.

Steps:

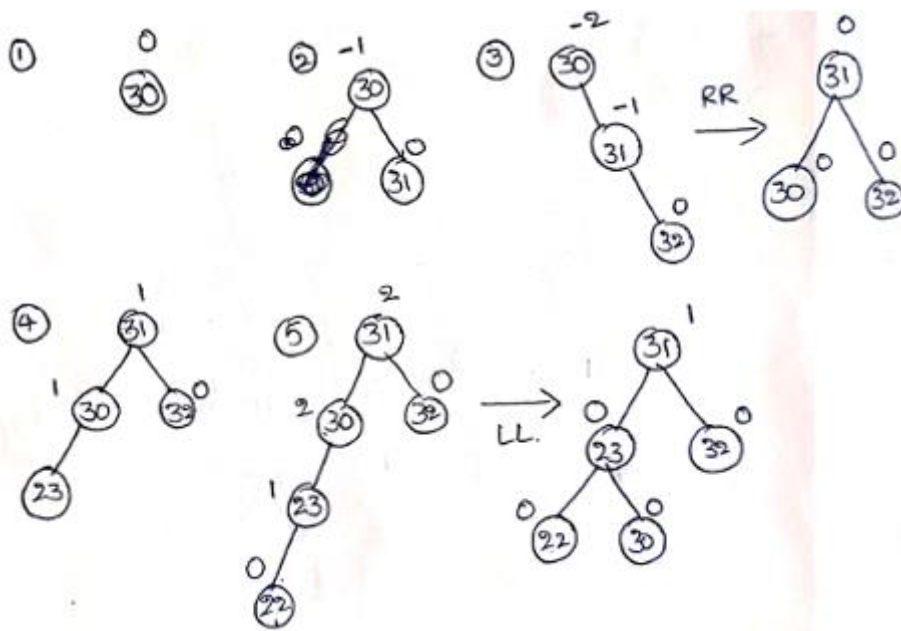
1. Perform LL Rotation
2. Perform RR Rotation

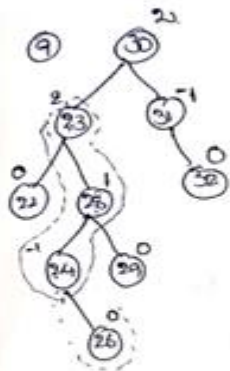
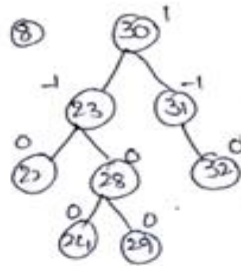
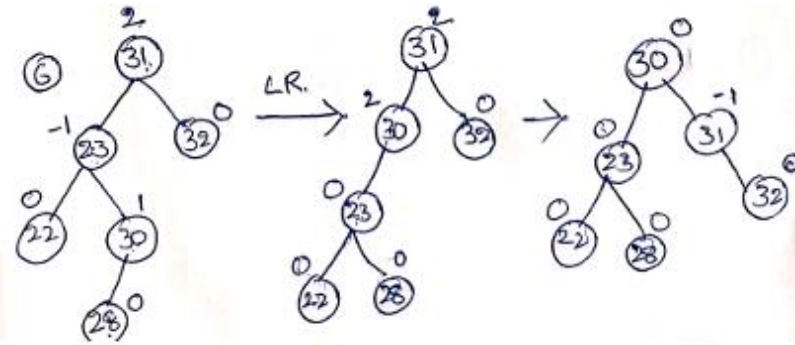


Insertion Algorithm:

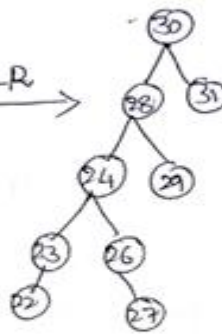
1. Insert a new node as new leaf node just as in ordinary binary search tree.
2. Starting from the inserted node and moving towards the root, check the BF of each node.
3. Identify the first unbalanced node, if any.
4. Apply the appropriate rotations to restore the balance of the AVL Tree,
5. After rotation, update the BF of the affected nodes.

Ex: Insert 30, 31, 32, 23, 22, 28, 24, 29, 26, 27, 34, 36

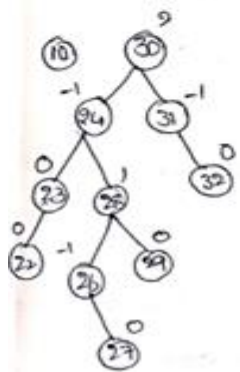
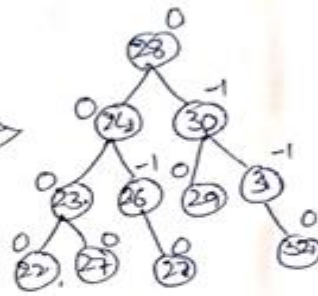




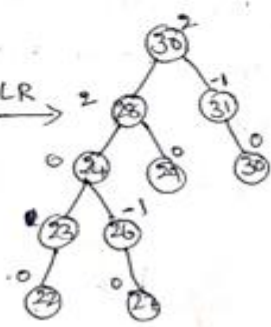
LR



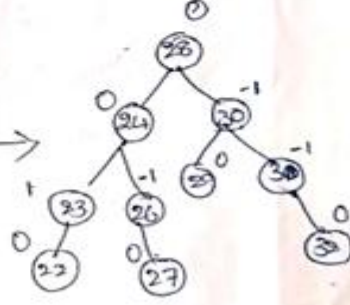
→



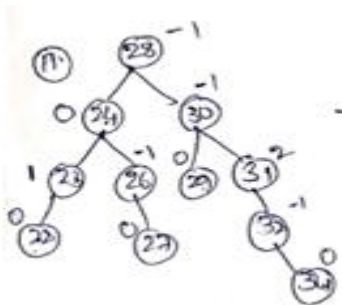
LR



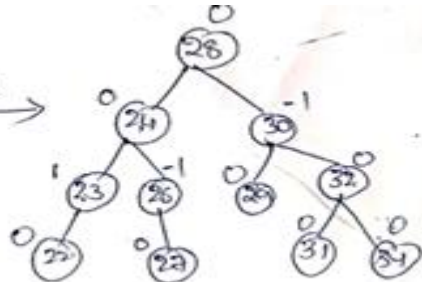
→

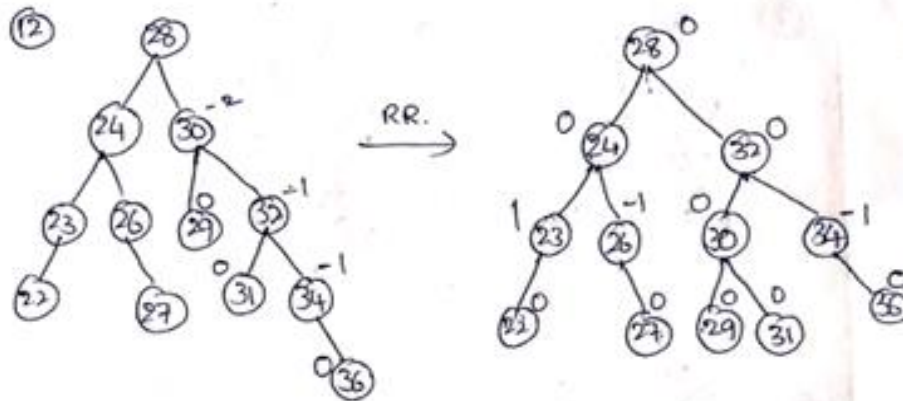


(i)



RR



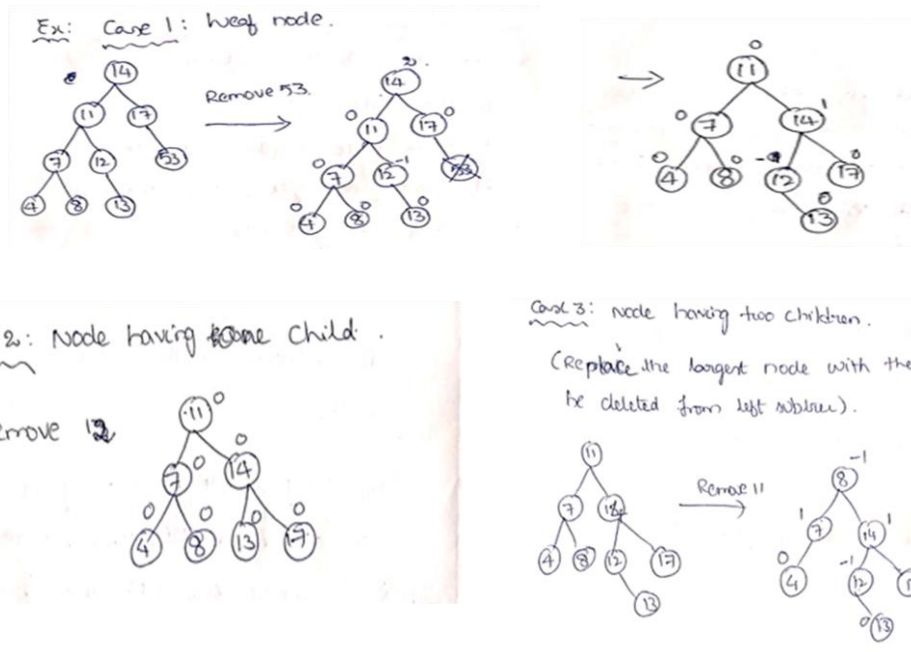


Exercise:

1. 3,2,1,4,5,6,7,16,15,14
2. 30,31,32,23,22,28,24,29,26,27,34,36
3. 7,14,2,5,10,33,56,30,15,25,66,70,4
4. 17,8,4,9,21,42,39,72,1,48,2,100,90,81
5. 9,7,28,32,49,94,11,4,19,52,10,70,90
6. 15,20,24,10,13,7,30,36,25
7. One, two, seven, nine, three, six, five, sixty, ten, thirty, ninety
8. March, May, November, August, April, January, December, July, February, June, October, September

Deletion Algorithm:

1. Search for the node to be deleted
2. If the node is;
 - Leaf node simply delete it.
 - Having one child, then the root node is deleted and the child node is brought forward to the parent.
 - Having two children, the next successive node on the right branch is used to replace the deleted node. The successive node is then deleted. The successive node is always the left most node on the right branch.
3. After deleting a node, there may be a chance for violation of BF. Whenever BF is violated apply appropriate rotations to make the unbalanced tree to balanced tree.



Searching:

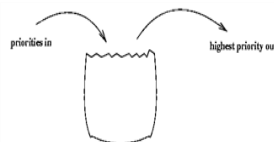
To search for a given key in the AVL tree;

1. Compare it with the root, if the key is found at the root return root.
2. If the key is less than the root, recur for the left subtree.
3. If the key is greater than the root, recur the search process in right subtree.

Heaps:

Priority Queue:

The priority queue is a data structure having the collection of elements which are associated with specific priority.



There are two types of priority queues. They are;

1. Ascending Priority Queue
2. Descending Priority Queue

Ascending Priority Queue:

It is a collection of elements in which the elements can be inserted arbitrary but only smallest element can be deleted.

Descending Priority Queue:

It is a collection of elements in which the elements can be inserted arbitrary but only largest element can be deleted.

Operations:

1. Insertion
2. Deletion
3. Searching for a particular element

Implementation: Priority queues can be implemented using arrays or linked list or heaps.

Applications:

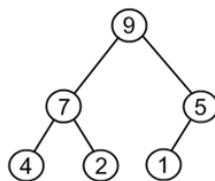
- Scheduling the jobs in OS
- Managing discrete events in simulation modeling
- In network communication to manage limited bandwidth for transmission

Binary Heaps:

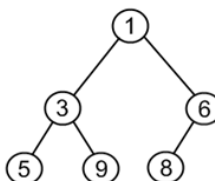
Heap is a complete binary tree or almost complete binary tree, where each node has higher priority than its children. This is called **heap-order** property. Each node has two children except for the last two levels. New nodes are inserted at the last level from left to right. This is called **heap-structure** property.

Heaps can be **Max Heap** or **Min Heap**.

Max Heap: A max heap is a tree in which value of each node is greater than or equal to the value of its child.



Min Heap: A min heap is a tree in which value of each node is less than or equal to the value of its child.



Insertion Algorithm:

1. Insert element at the last position in heap.
2. Compare with its parent and swap if the heap – property violates.
3. Continue comparing the new element with nodes towards up each time for satisfying the heap - property.

Ex: Consider the following elements and insert into a tree. (Max Heap)

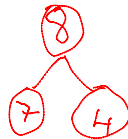
8, 7, 4, 26, 28, 34, 1, 3, 15, 9

Insert 8: 

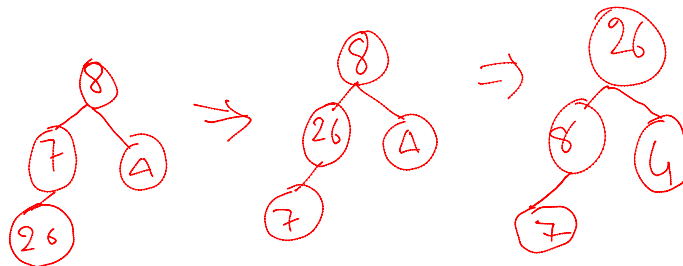
Insert 7:



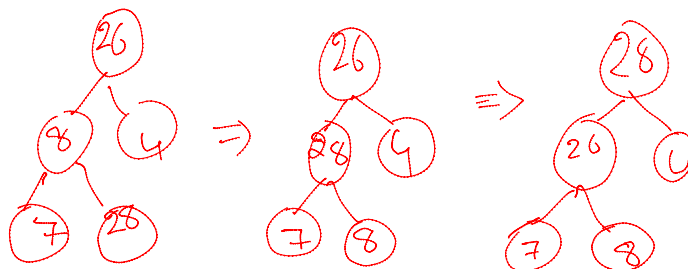
Insert 4:



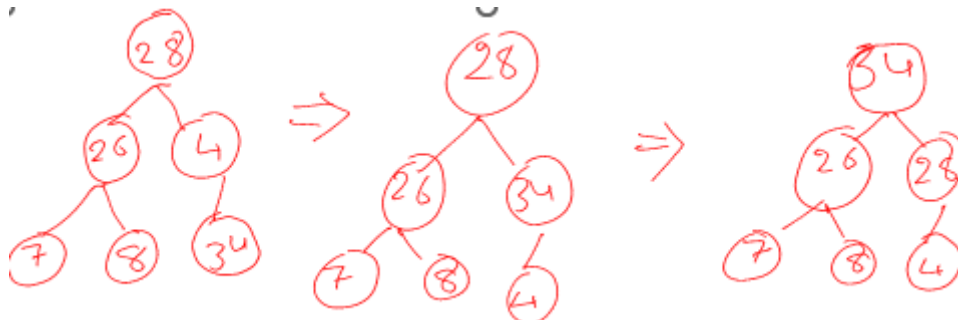
Insert 26:



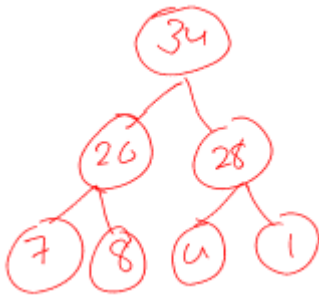
Insert 28:



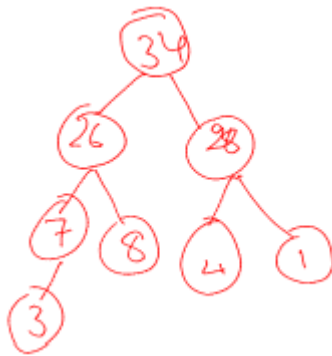
Insert 34:



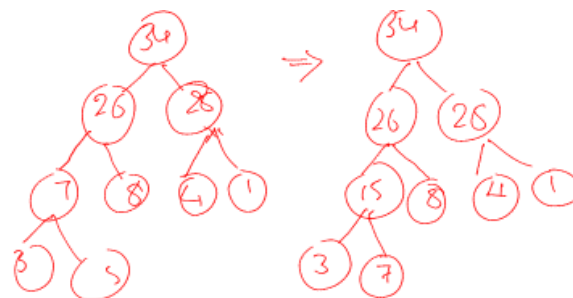
Insert 1:



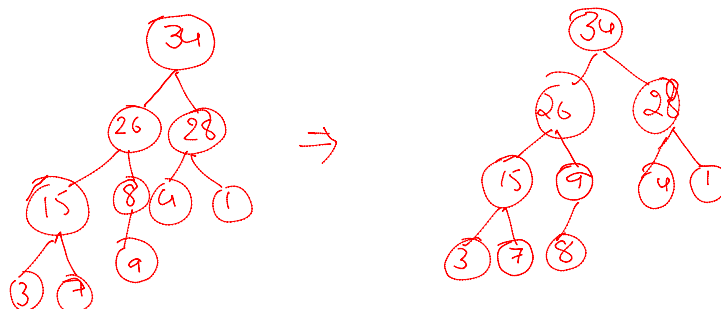
Insert 3:



Insert 15:



Insert 9:



Exercise:

Construct Min Heap for the following elements.

8, 7, 4, 5, 6, 2, 3, 11, 9

Construct a Min Heap for the following.

5, 9, 4, 2, 1, 6

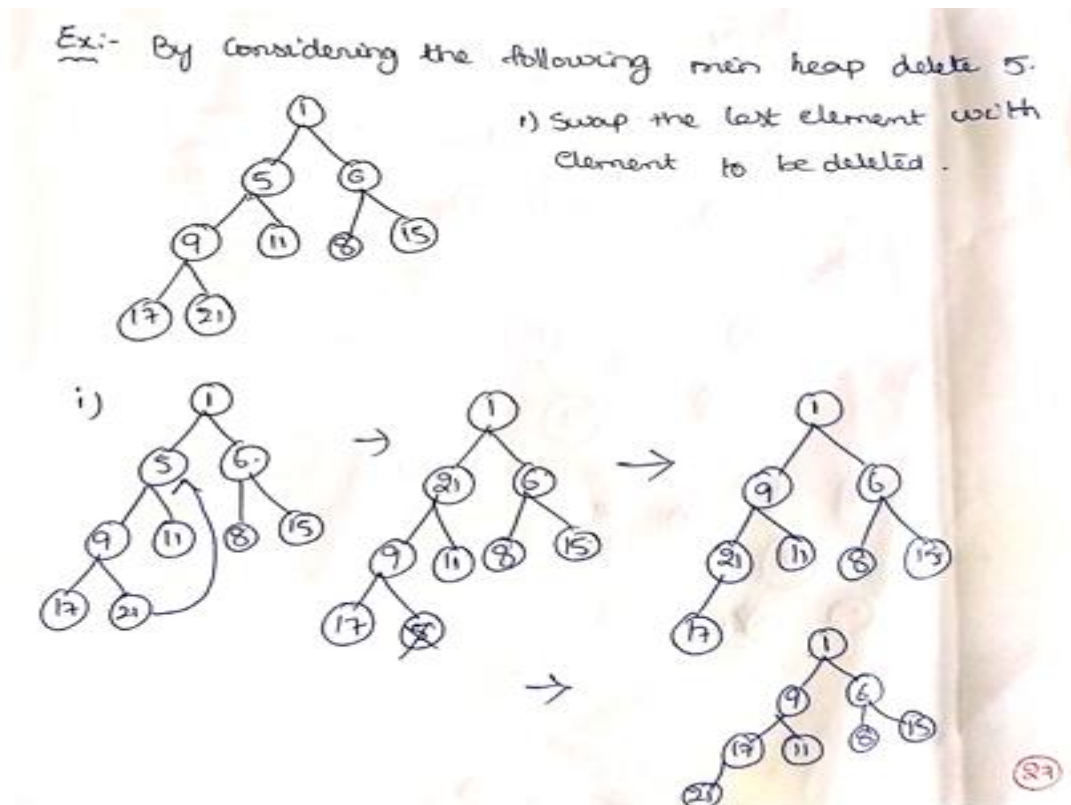
Construct a Min Heap for the following.

2, 5, 8, 1, 7, 25, 48, 3, 49, 6, 4, 9.

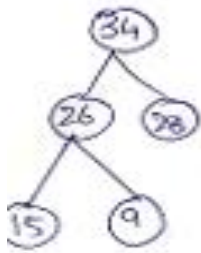
Deletion Algorithm:

1. Exchange the root with last leaf.
2. Decrease the heap size by 1.
3. Heapify the minimum element (min heap)/ maximum element (max heap) using bottom up construction algorithm and also each time we have maintain heap property.

Ex:



Ex: Delete (heapify) the element from max heap.



1. Swap root with leaf.
2. Delete the element.
3. Check heap order property, swap nodes if any violation.
4. Continue the process until all elements are deleted.

34	28	26	15	9
----	----	----	----	---

